

Order Identifiers

What each id is, where it comes from, and why it exists

Trading Platform — risk engine reference

Overview

By the time an order is committed, it carries **four** distinct identifiers. They are not one id reused four ways. Each is created in a different place, by a different component, and answers a different question:

Identifier	Answers	Minted by
<code>client_order_id</code>	“Which <i>request</i> is this?”	the caller, before sending
<code>order_id</code>	“Which <i>order</i> is this, in our system?”	the C++ engine, at commit
<code>event_id</code>	“Which <i>commit event</i> is this?”	the C++ engine, at commit
<code>account_sequence</code>	“Where does this fill sit in the account’s history?”	the C++ engine, per-user counter

Why four, and not one

The code before `ExecuteOrder` effectively had one id. The client sent an `order_id`, Python threw it away and generated its own UUID, and there was no stable key that survived a retry. Two consequences:

- **No duplicate detection.** `orders.idempotency_key` existed in the schema with a unique constraint, but nothing ever wrote to it. A client that timed out and retried placed the order twice.

- **Wrong error reporting.** `checkOrder` returned `approved: false` plus a free-text reason that Python never read, so every rejection — bad limit price, unknown symbol, genuinely insufficient funds — was reported to the user as “insufficient funds, requires o”.

Splitting the identity into four named things fixes both: a stable `client_order_id` makes retries recognisable, and the engine returning a typed result (keyed to the right identifier) makes every rejection reason distinct.

`client_order_id` — identifies the request

What it is. An opaque string chosen by whoever places the order, *before* the request is sent. It is stable across retries: if a client sends an order, times out, and sends it again, both requests carry the same `client_order_id`. One logical order has exactly one `client_order_id` for its whole life.

Where it’s created.

- `strategy_runner.py` generates `str(uuid.uuid4())` for each order intent.
- An API client puts it in the request body (currently the field is named `order_id`).

How it flows. Untouched, end to end:

caller

```
→ POST /api/v1/orders                body {"order_id": "..."}
→ order_routes.place_order           order_id = data["order_id"]
→ OrdersService.place_order(order_id=...)
→ RiskEngineClient.execute_order(client_order_id=order_id)
→ ExecuteOrderRequest.client_order_id (gRPC)
→ UserStateStore::execute_order(..., client_order_id, ...)
  · key into the per-user idempotency cache
→ WALwriter::write_fill(..., client_order_id, ...) → wal.log
→ fill_consumer.process_fill →
```

`orders.idempotency_key`

Why it exists. It is the idempotency key. When an order fills, the engine stores the result in a per-user map keyed by `client_order_id`. A second call with the same key:

- **same order parameters** → the engine returns the *original* result without executing anything. The caller gets the same 201 FILLED, no double charge.
- **different parameters** → the engine returns `IDEMPOTENCY_CONFLICT` (surfaced as HTTP 409). The caller reused an id for a different order, which is a bug on their side, and the engine refuses to guess which one they meant.

Downstream, `fill_consumer` writes it to `orders.idempotency_key`, whose unique constraint is a second, cross-process line of defence.

order_id — identifies the order in our system

What it is. The canonical server-side id for the order. 32 random hex characters, generated by the engine.

Where it's created. In `UserStateStore::execute_order`, the moment the order commits:

```
res.order_id = random_id(); // 32 hex chars from a per-thread PRNG
```

How it flows. Written to the WAL line, returned in the `ExecuteOrder` response, then used as `order.id` — the primary key — on both the transient object the API returns and the row `fill_consumer` writes. The two always agree because they use the same value.

Why it exists. Two reasons:

1. **A name we control.** The client's `client_order_id` is arbitrary and only unique *per user* (two users could both send "order-1"). `order_id` is a single global namespace we own, suitable for a primary key and for referencing the order from other tables (`risk_checks`, `ledger_entries`).
 2. **WAL-replay detection.** `fill_consumer` checks `order_id` against the primary key first. If `wal_producer` crashes after sending a line to Kafka but before saving its file cursor, it resends that line on restart — same `order_id`, caught here, skipped.
-

event_id — identifies one commit event

What it is. 32 random hex characters identifying a single act of committing a fill — as opposed to the order it belongs to.

Where it's created. Right next to `order_id` in `execute_order`:

```
res.event_id = random_id();
```

How it flows. Written to the WAL, carried in the response, read by `fill_consumer`. It has no database column yet.

Why it exists. One order can, in a specific failure scenario, produce **two** commit events:

1. Client places order with `client_order_id = x`. Engine executes it, assigns `order_id = A`, `event_id = P`, commits, replies.
2. The reply is lost (network); the engine then restarts and loses its in-memory idempotency cache.
3. Client retries with the same `client_order_id = x`. The engine has no memory of it, so it executes again — `order_id = B`, `event_id = Q`.

Now there are two orders (A, B) for one intent. The `client_order_id` check in `fill_consumer` and the `idempotency_key` unique constraint stop B from being persisted. `event_id` is the identifier that lets us tell the two commit events apart in logs and, later, in an audit table — `order_id` alone can't, because A and B are different.

account_sequence — orders one account's fills

What it is. A `uint64` counter, one per user, that only ever increases. User 42's first fill is 1, their second is 2, and so on.

Where it's created. In `execute_order`, under the user's lock, at commit:

```
user.sequence += 1;
res.account_sequence = user.sequence;
```

How it flows. Written to the WAL, carried in the response, read by `fill_consumer`. No column yet.

Why it exists. It makes the *order* of an account's fills explicit and verifiable. Because the WAL write happens while the per-user lock is still held, the order of lines in `wal.log` for a given user can never diverge from their sequence numbers.

A consumer can then check: "I last applied user 42's sequence 5; this message is sequence 7 — sequence 6 is missing or delayed." Right now the Kafka topic has a single partition, so ordering is guaranteed for free. The moment it is split into multiple partitions for throughput, two fills for one user could be delivered out of order — and since `fill_consumer` writes the *absolute* new balance from each event (not a delta), applying them backwards leaves the wrong final balance. `account_sequence` is the groundwork for detecting and rejecting that (GitHub issue #24).

Where they enter the durable record

All four identifiers are written into the WAL line by a single call inside `execute_order`, while the per-user lock is held:

```
wal->write_fill(user_id, res.order_id, symbol, side, order_type,
               quantity, fill_price,
               res.new_cash, res.new_quantity, res.new_average_price,
               client_order_id, res.event_id, res.account_sequence);
```

Everything after this point — `wal_producer.py` → Kafka `order-fills` → `fill_consumer.py` → Postgres — is a replay of that line. If an identifier is not in the line, it does not exist downstream.

Duplicate detection, layer by layer

Layer	Key it uses	Failure it catches
fill_consumer check 1	order_id (primary key)	WAL replay — wal_producer resends a line after crashing between “sent to Kafka” and “saved cursor”
fill_consumer check 2	client_order_id → orders.idempotency_key	engine re-execution — — engine restarted, lost its cache, client retried; the new row has a fresh order_id but the same client key
orders.idempotency_key unique constraint + except IntegrityError	client_order_id	consumer race — two consumer processes get past both checks at the same instant; the database rejects the second commit()

The engine’s own in-memory idempotency cache is the first line of defence and handles the common case (a fast retry while the engine is still up) without any of this ever running.